



Government  
of Canada

Gouvernement  
du Canada

## ESDC Development Community

# Application Logging

*Le texte français est donné à la suite.*

## Recommendation

**Scope of the recommendation:** Application Logging

**Out of Scope:** IIS Logging, Auditing

**Definition:** An application log is a file of events that are recorded by a software application. It contains errors, informational events and warnings. The format and content of an application log are determined by the developer of the software program, rather than the operating system.

**Usage:** Logging/Recording application events to accommodate trouble shooting, provide health status (for the application) or preventing an issue from becoming a larger problem.

**Alternative use:** Alerting technical groups of specific event when it occurs.

## Top 5 Best Practices

- Use a standard and easily configurable logging framework. [NLog](#), [log4j 2](#), [log4net](#), [ELMAH](#), etc. allow faster config changes than hard-coded or proprietary frameworks.
- Use a logging framework with flexible output options. View console logs in development and centralize prod logs without extra plugins or agents.

- Don't let logging block your application. Write logs asynchronously with a buffer or queue so the application can keep running.
- Offer a standard logging configuration for all teams. Avoid chaos as the company grows. Start with a best practice and let teams deviate as needed.
- Don't forget legacy application logs. Find a way to send logs from legacy apps, which are frequently culprits in operational issues.

## What to log

Debug logs typically report application events that are useful when diagnosing a problem. Investigations into application failures need the “W” words: Who, What, When, Where and Why:

- Who was using the system when it failed?
- Where in the code did the application fail?
- What was the system doing when it failed?
- When did the failure occur?
- Why did the application fail?

## Suggested Log Types

- **Fatal:** An unexpected error/exception that compromises the ability of the application to function properly for further requests. (for example: “An exception occurred loading application initialization data, application will not be available”)
- **Error:** An error/exception in the execution that, while it may have been reported to the user and/or be benign, should be in the log for troubleshooting reasons. Also useful if the developer specifically wants an error to be logged to keep track of unusual situations.

Validation/data entry errors would not normally be part of that. (for example: “Contacting backup server failed, saving transaction as pending.”)

- **Warning:** Not necessarily an error for the end user but an unusual situation/condition occurred (for example, “Contacting primary server failed, attempting to contact backup server”)
- **Information:** A normal situation/condition that is useful to have reported in the log (for example: “Lookup data loaded in cache, expiring in 12 hours”)
- **Debug:** Details the great lines of whatever functions executes. As the name suggests, is useful for debugging purposes (for example “Lookup data found in cache, skipping loading from file”)
- **Trace:** Like Debug, but at a much finer level of details. Messages at Trace level can often match the actually steps pseudo-code of the algorithm (for example: “Checking if user is of correct type”, “Setting up call to user service”)

## How to log

Avoid implementing any in-house logging mechanism. Existing library are very mature and offer a wide range of granularities and configuration options. We recommend the use of existing Logging libraries

- NLog for the .NET development environments
- SLF4J API and Log4j 2 for the Java development environment
- Elmah (Error Logging Module and Handlers for ASP.NET) - Open source

# When to Log

In all verticals using appropriate log levels based on the analysis needs

- Dev, INT, UAT, PRF: Higher granularity of logs
- Production - regular production situation: Lower granularity
- Production - Production issue analysis: Use a progressive increase in the granularity until issue is understood.

## Where to send log data

**At the moment there is no setup available to send our logs. We think that this could be a very good initiative for IITB.**

The relational databases are not the best place to send log data. Time-series databases (TSDB) are much more efficient at storing log data. Open-source TSDB such as InfluxDB are much better suited to storing log data than relational databases.

The ELK stack is a popular solution for log aggregation. ELK is an acronym that stands for Elasticsearch, Logstash, and Kibana.

Elasticsearch is a fast search engine that is used to find data in large datasets.

Logstash is a data pipeline platform that will collect log data from many sources and feed it to a single persistence target.

Kibana is a web-based data visualizer and search engine that integrates with Elasticsearch.

## Log File Location

The following locations are set by SADE and are the departments standard

- For all applications in Non-Production:

```
E:\\WebLogs\\[SADE-Vertical]\\[application abbreviation]\\[application sub folders if required]
```

- For all application in production:

```
E:\\WebLogs\\[application abbreviation]\\[application sub folders if required]
```

**Note from SADE:** "If the log directory structure does not exist as stated above, the application should take the initiative to create it (on the fly). Understanding that the directory name and structure should be defined in the Web.Config"

## Log File Content

### File name

- 1 file per day: - Easier to archive; - Size stays small (easier to load, gives quick idea if you received more errors on a particular day)
- File name should include: the applications components name, date and server name (makes it easier when bringing those files to your local pc and knowing what they are)

Ex: APP\_NAME\_20190930\_MLWB200.log

**Content Data will be determined by the requirement and need. We recommend including the following data elements for each log record:**

- Date/Time
- Web Service Name
- Server Name

- Full Name
- AppDomainName
- Thread Name
- Caller
- Message
- «more details»

## **Sensitive content**

- Protected B data elements should not be part of the log record. If needed, Protected B data should be encrypted.

# **How do you access the logs?**

For apps hosted in Intranet, use Shared Folders to the log file location set by SADE.

For apps hosted in OPS or DMZ (Internet), the accessibility of the log files is on request only. 2 options exist:

- Open a ticket with the National Service Desk (NSD), to be processed by iNET, requesting a copy of the specific log file. This will provide a copy of the current file (up to the minute)).
- It is also possible to request a regular transfer (daily, twice daily, weekly, ...). This process must be configured with the EDS, in collaboration with iNET. This will provide log files from the previous day.

---

*Texte français:*

## Recommandation

**Portée de la recommandation :** Journalisation des applications

**Hors portée :** Journalisation IIS, Audit

**Définition :** Un journal d'application est un fichier d'événements enregistrés par une application logicielle. Il contient des erreurs, des événements d'information et des avertissements. Le format et le contenu d'un journal d'application sont déterminés par le développeur du logiciel plutôt que par le système d'exploitation.

**Utilisation :** Journaliser/enregistrer les événements d'application pour faciliter le dépannage, fournir l'état de santé (de l'application) ou éviter qu'un incident ne s'aggrave.

**Autre utilisation :** Alerter les groupes techniques lorsqu'un événement spécifique survient.

## 5 meilleures pratiques

- Utilisez un cadriciel de journalisation standard et facilement configurable. NLog, log4j 2, log4net, ELMAH, etc., permettent des modifications de configuration plus rapides que les cadriciels codés en dur ou propriétaires.
- Utilisez un cadriciel de journalisation avec des options de sortie flexibles. Affichez les journaux dans la console en développement et centralisez les journaux de production sans modules d'extension ou agents supplémentaires.
- Ne laissez pas la journalisation bloquer votre application. Écrivez les journaux de manière asynchrone avec une mémoire tampon ou une file d'attente afin que l'application continue de fonctionner.
- Offrez une configuration de journalisation standard pour toutes les équipes. Évitez le désordre au fur et à mesure que l'organisation

grandit. Commencez par une bonne pratique et laissez les équipes s'en écarter au besoin.

- N'oubliez pas les journaux des applications patrimoniales. Trouvez un moyen de transmettre les journaux des applications existantes, qui sont fréquemment la cause de problèmes opérationnels.

## Quoi journaliser

Les journaux de débogage rapportent généralement des événements d'application utiles pour diagnostiquer un problème. Les enquêtes sur les défaillances d'application nécessitent de répondre aux questions : Qui, Quoi, Quand, Où et Pourquoi :

- Qui utilisait le système lorsqu'il est tombé en panne ?
- Où dans le code l'application a-t-elle échoué ?
- Que faisait le système lorsqu'il est tombé en panne ?
- Quand la défaillance s'est-elle produite ?
- Pourquoi l'application a-t-elle échoué ?

## Types de journaux suggérés

- **Fatal (Critique)** : Une erreur/exception inattendue qui compromet la capacité de l'application à fonctionner correctement pour les requêtes subséquentes. (par exemple : « Une exception s'est produite lors du chargement des données d'initialisation, l'application ne sera pas disponible »)
- **Error (Erreur)** : Une erreur/exception dans l'exécution qui, bien qu'elle ait pu être signalée à l'utilisateur et/ou être bénigne, doit figurer dans le journal à des fins de dépannage. Également utile si le développeur souhaite spécifiquement enregistrer une erreur pour suivre des situations inhabituelles. Les erreurs de validation ou de saisie de

données n'en font normalement pas partie. (par exemple : « Échec de la connexion au serveur de secours, enregistrement de la transaction en attente »)

- **Warning (Avertissement)** : Pas nécessairement une erreur pour l'utilisateur final, mais une situation ou condition inhabituelle s'est produite (par exemple, « Échec de la connexion au serveur principal, tentative de connexion au serveur de secours »)
- **Information** : Une situation ou condition normale qu'il est utile de consigner dans le journal (par exemple : « Données de référence chargées dans la mémoire cache, expiration dans 12 heures »)
- **Debug (Débogage)** : Détaille les grandes lignes de l'exécution des fonctions. Comme son nom l'indique, c'est utile à des fins de débogage (par exemple, « Données trouvées dans la mémoire cache, omission du chargement depuis le fichier »)
- **Trace** : Semblable au débogage, mais à un niveau de détail beaucoup plus fin. Les messages au niveau Trace correspondent souvent aux étapes réelles du pseudo-code de l'algorithme (par exemple : « Vérification si l'utilisateur est du type approprié », « Configuration de l'appel au service utilisateur »)

## Comment journaliser

Évitez d'implémenter des mécanismes de journalisation maison. Les bibliothèques existantes sont très matures et offrent une grande variété de granularités et d'options de configuration. Nous recommandons l'utilisation de bibliothèques de journalisation existantes :

- NLog pour les environnements de développement .NET
- SLF4J API et Log4j 2 pour l'environnement de développement Java

- Elmah (Error Logging Module and Handlers for ASP.NET) - Source ouverte

## Quand journaliser

Dans toutes les branches applicatives en utilisant des niveaux de journalisation appropriés selon les besoins d'analyse :

- Dev, INT, UAT, PRF : Granularité plus élevée des journaux
- Production - situation normale : Granularité plus faible
- Production - analyse d'un problème : Augmenter progressivement la granularité jusqu'à ce que le problème soit compris.

## Où envoyer les données de journalisation

**Pour le moment, aucune infrastructure n'est disponible pour centraliser nos journaux. Nous pensons que cela pourrait être une excellente initiative pour la DGIIT.**

Les bases de données relationnelles ne sont pas le meilleur endroit pour envoyer des données de journaux. Les bases de données de séries temporelles (TSDB) sont beaucoup plus efficaces pour stocker ces données. Les TSDB à code source ouvert telles qu'InfluxDB sont bien mieux adaptées au stockage de données de journaux que les bases de données relationnelles.

La suite ELK est une solution populaire pour l'agrégation de journaux. ELK est un acronyme signifiant Elasticsearch, Logstash et Kibana.

Elasticsearch est un moteur de recherche rapide utilisé pour trouver des données dans de grands ensembles de données.

Logstash est une plateforme de pipeline de données qui recueille des données de journaux provenant de nombreuses sources et les transmet à

une cible de persistance unique.

Kibana est un outil de visualisation de données et un moteur de recherche web qui s'intègre à Elasticsearch.

## Emplacement des fichiers journaux

Les emplacements suivants sont définis par la SADE et constituent la norme ministérielle :

- Pour toutes les applications hors production :

```
E:\\WebLogs\\[SADE-Vertical]\\[abréviation application]\\[sous-dossiers application au besoin]
```

- Pour toutes les applications en production :

```
E:\\WebLogs\\[abréviation application]\\[sous-dossiers application au besoin]
```

**Note de la SADE :** « Si la structure de répertoires des journaux n'existe pas telle qu'indiquée ci-dessus, l'application doit prendre l'initiative de la créer (à la volée). Il est entendu que le nom et la structure du répertoire doivent être définis dans le Web.Config »

## Contenu du fichier journal

### Nom de fichier

- 1 fichier par jour : - Plus facile à archiver; - La taille reste réduite (plus facile à charger, donne un aperçu rapide si vous avez reçu plus d'erreurs un jour précis)
- Le nom de fichier doit inclure : le nom des composants de l'application, la date et le nom du serveur (facilite l'identification lors du rapatriement sur votre poste local)

Ex : APP\_NAME\_20190930\_MLWB200.log

**Les données du contenu seront déterminées selon les exigences et les besoins. Nous recommandons d'inclure les éléments de données suivants pour chaque enregistrement de journal :**

- Date/Heure
- Nom du service Web
- Nom du serveur
- Nom complet
- AppDomainName
- Nom du fil d'exécution (Thread)
- Appelant
- Message
- « plus de détails »

### **Contenu sensible**

- Les éléments de données de niveau Protégé B ne doivent pas faire partie de l'enregistrement de journal. Au besoin, les données de niveau Protégé B doivent être chiffrées.

### **Comment accéder aux journaux ?**

Pour les applications hébergées sur l'Intranet, utilisez les dossiers partagés vers l'emplacement des fichiers journaux défini par la SADE.

Pour les applications hébergées en OPS ou en DMZ (Internet), l'accès aux fichiers journaux se fait uniquement sur demande. 2 options existent :

- Ouvrir un billet auprès du Centre de services national (NSD), qui sera traité par iNET, demandant une copie du fichier journal spécifique. Cela fournira une copie du fichier actuel (à la minute près).

- Il est également possible de demander un transfert régulier (quotidien, deux fois par jour, hebdomadaire, ...). Ce processus doit être configuré avec les EDS, en collaboration avec iNET. Cela fournira les fichiers journaux de la veille.